

SCHOOL OF DIGITAL TECHNOLOGY AND COMPUTING

BCS2023

Computer Science Theory

GROUP ASSIGNMENT 2 + PRESENTATION

Group No.	
Scenario Chosen (A – E or own)	
Submission Date	

Group Members & Contribution Declaration

Each member must enter their contribution percentage and sign. Percentages must total 100%.

No.	Student Name	Student ID	Contribution (%)	Signature
1				
2				
3				
4				
Total			100 %	

3. Student Declaration

We, the undersigned, declare that this assignment is the original work of our group and that all external sources have been acknowledged in APA 7th edition format. We confirm that the contribution percentages stated above are a true and accurate reflection of each member's work, and that all members participated in both Part A and Part B. We understand that plagiarism, collusion, or undeclared AI-generated content may result in zero marks and disciplinary action under university regulations.

Group Leader's Signature: _____ Date: _____

4. Marks (Fill in Your Student ID & Name ONLY)

No.	Student Name	Student ID	Report (/15)	Presentation (/5)	Total (/20)
1					
2					
3					
4					

PART A: REQUIREMENT ANALYSIS AND SOLUTION DESIGN

A.1 Requirement Analysis

A.1.1 Functional Requirements

The Smart Community Service Management System must provide the following core functions to support both residents (users) and community centre staff (administrators):

No.	Functional Requirement	Description	Justification / Importance
FR1	User Registration & Login	Allows residents to create an account and securely log in.	Ensures only verified users can register for activities and protects personal data (name, IC/contact, participation history).
FR2	View Available Activities	Displays a list/catalogue of volunteer programmes, workshops, health campaigns and social events with details (date, venue, slots).	Gives users a single, centralised place to browse community activities instead of checking multiple platforms.
FR3	Register for Activities	Allows a logged-in user to sign up for an activity and receive confirmation.	Directly automates the core manual process of the community centre, replacing paper/ad-hoc sign-ups.
FR4	View Participation History	Allows a user to view a personal record of activities joined in the past.	Provides transparency and helps the community centre and users track engagement over time.
FR5	View Schedule & Announcements	Displays upcoming activity schedules and centre announcements.	Keeps users informed and reduces missed or clashing events.
FR6	Admin – Manage Activities	Allows an administrator to create, update, and remove activity records (CRUD).	Enables staff to keep activity information accurate and up to date without developer intervention.
FR7	Admin – View Registrations & Generate Reports	Allows an administrator to view who has registered for each activity and export/generate a basic participation report.	Supports planning, resource allocation, and accountability for the community centre.

Together, these functions replace the community centre's current practice of managing activities across multiple disconnected platforms with a single, centralised web-based system, directly satisfying the project scenario.

A.1.2 Non-Functional Requirements

Beyond core functionality, the system must satisfy the following quality attributes to be effective and trustworthy in real use:

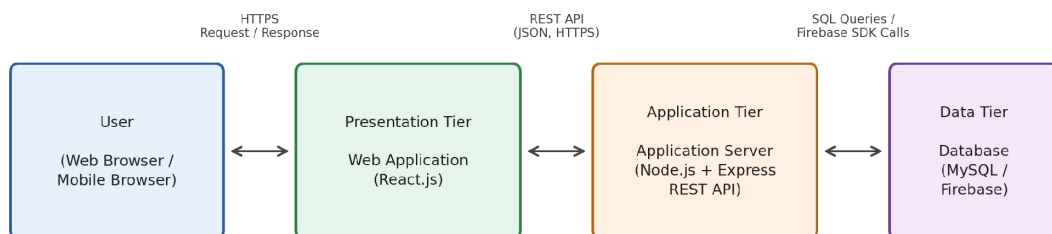
Non-Functional Requirement	Description	Justification / Importance
Security	User data (login credentials, personal details, participation records) must be protected through authentication, authorization, and encrypted data transmission (HTTPS).	The system stores personal and registration data; a breach could expose residents' private information and damage trust in the community centre.
Usability	The interface must be simple, intuitive, and accessible to users of varying technical ability, including elderly or non-technical residents.	The system serves the general public, so ease of use directly affects adoption; a confusing interface would push users back to manual processes.
Performance	The system should respond to common actions (login, browsing activities, registering) within a few seconds under normal load.	Slow response discourages use, especially during peak registration periods for popular events.
Reliability	The system should be available and function correctly with minimal downtime, especially during activity registration periods.	Downtime during a registration window could cause residents to miss out on limited activity slots.
Scalability	The architecture should support an increasing number of users and activities as the community centre grows.	As more programmes are added and more residents adopt the platform, the system must handle increased load without redesign.

A.2 Distributed Solution Design

A.2.1 System Architecture

The proposed system follows a three-tier client-server architecture consisting of a Presentation Tier, an Application Tier, and a Data Tier, as illustrated in Figure 1 below.

Figure 1: Three-Tier Client-Server Architecture of the Smart Community Service Management System



Component overview:

- User: Residents and administrators access the system through a standard web browser on a desktop or mobile device.

- Web Application (Presentation Tier – React.js): Renders the user interface (login, activity listing, registration forms, dashboards) and communicates with the Application Server through HTTPS REST API calls.
- Application Server (Application Tier – Node.js + Express): Hosts the business logic, handles authentication and session management, validates input, and exposes REST API endpoints consumed by the React frontend.
- Database (Data Tier – MySQL or Firebase): Persistently stores user accounts, activities, registrations, and schedules, and responds to queries from the Application Server.

Each tier can be developed, deployed, and scaled independently, and communication between tiers occurs over a network (HTTP/HTTPS), which is the defining characteristic of a distributed application as opposed to a single monolithic desktop program.

A.2.2 Distributed Computing Approach

The team has selected a Three-Tier Client-Server Architecture (a specific form of web-based distributed application) as the distributed computing approach for this system.

Justification for this choice:

- Separation of concerns: Presentation (React), logic (Node.js/Express), and data (MySQL/Firebase) are decoupled, making the system easier for a group of students to divide, develop, and test in parallel.
- Centralised data management: A single application server and database ensure that all users see consistent, up-to-date activity and registration information, which is essential for a shared community resource.
- Multi-user, network-based access: Client-server (as opposed to a two-tier or peer-to-peer model) naturally supports many residents accessing the same activities and schedules concurrently from different devices and locations.
- Suitability for project scope: A three-tier web architecture is well within the complexity appropriate for a "small" distributed application (per CLO3), while still clearly demonstrating client-server communication, database connectivity, and CRUD operations across a network (per CLO2).

A.2.3 Technology Stack

The following technologies have been selected for the implementation of the system:

Layer	Technology	Justification
Frontend	React.js (with React Router, Axios/Fetch)	Component-based structure makes it easy to build reusable UI elements (activity cards, forms, dashboards); large ecosystem and strong community support; efficient rendering via the virtual DOM suits a data-driven interface that updates frequently (activity lists, registration status).
Backend	Node.js with Express.js	JavaScript on both frontend and backend reduces context-switching for the team and speeds up development; Express simplifies building RESTful APIs for authentication, activity management, and registration;

		Node's non-blocking I/O suits a multi-user, request-heavy web application.
Database	MySQL (relational) or Firebase (cloud/NoSQL)	MySQL is chosen as the primary option because the data is naturally relational (Users, Activities, Registrations, Schedules have clear foreign-key relationships), which supports data integrity via primary/foreign keys. Firebase is listed as an alternative for its built-in authentication, real-time database, and easy cloud hosting if the team prefers a serverless setup with less server configuration.
UI/UX Design	Figma	Enables the team to collaboratively design and review wireframes/prototypes for the Login, Registration, Activity Listing and Dashboard pages before development, ensuring the interface is planned and consistent.
Supporting Tools	Visual Studio Code, Git/GitHub, Postman, XAMPP/MySQL Workbench (if MySQL is used)	VS Code for development; Git/GitHub for version control and group collaboration; Postman for testing REST API endpoints; MySQL Workbench/XAMPP for managing the database schema.

Overall, this stack (React + Node.js + MySQL/Firebase, designed in Figma) was chosen because it uses a single language (JavaScript) across the frontend and backend, has strong documentation and community support for a student project, and directly demonstrates the client-server / three-tier distributed computing concepts required by the assignment, while remaining light enough to complete within the project timeline.

PART B: SYSTEM DESIGN AND DATABASE DESIGN (PLANNING)

B.1 User Interface Plan

The system requires eleven (11) key interfaces in total: six (6) for the User module and five (5) for the Admin module. This satisfies the minimum of four (4) wireframes required by the assignment, while covering all functional requirements identified in Part A.

ID	Page / Interface	Module	Purpose / Key Elements
P1	Login Page	User	Email/username + password fields, "Login" button, link to Registration page, role-based redirect (User → Activity Listing, Admin → Admin Dashboard).
P2	Registration (Sign Up) Page	User	Form to create a new account: name, email, phone, password; redirects to Login on success.
P3	Activity Listing Page (User Home)	User	Browsable/searchable list of activities (cards or table) with category filter; entry point after login for normal users.
P4	Activity Details & Registration Page	User	Full details of a selected activity (description, date, venue, slots left) with a "Register" button and confirmation message.
P5	My Participation History Page	User	List of activities the logged-in user has registered for, past and upcoming, with status (registered/attended/cancelled).
P6	Schedule & Announcements Page	User	Calendar/list view of upcoming activity schedules plus centre announcements/notices.
P7	Admin Dashboard	Admin	Landing page after admin login; summary widgets (total activities, total registrations, upcoming events) and navigation to admin functions.
P8	Manage Activities Page	Admin	Table listing all activities with Edit/Delete actions and an "Add Activity" button (CRUD entry point).
P9	Add / Edit Activity Form Page	Admin	Form to create or update an activity's title, description, category, capacity, and status.
P10	Manage Schedules Page	Admin	Table listing schedule sessions per activity with Add/Edit/Delete actions.
P11	View Registrations & Reports Page	Admin	Table of registrations per activity (who registered) with an option to generate/export a basic participation report.

B.1.1 Navigation Flowchart

The flowchart below shows how the eleven pages connect and how a User or Admin moves between them after logging in.

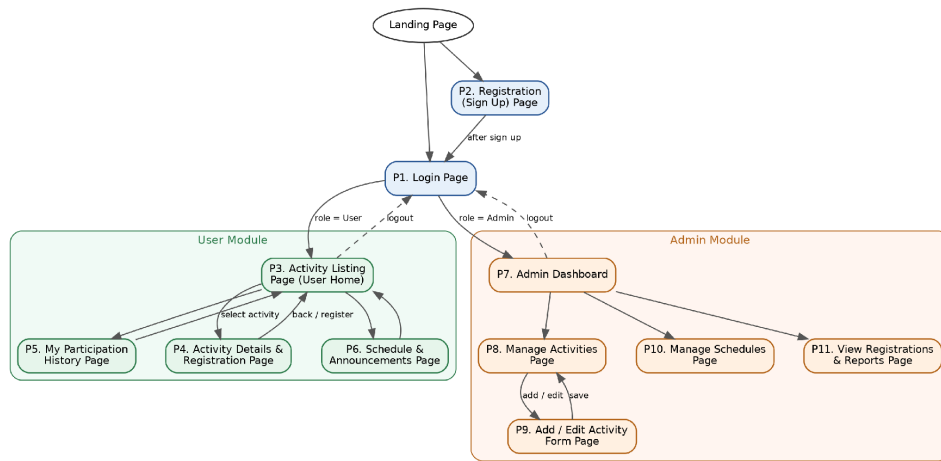


Figure 2: Navigation Flow / Site Map of the Smart Community Service Management System

Flow summary:

- All users start at the Landing Page and either Log In or Register; after registration they are sent to Login.
- On login, the system checks the user's role: normal users are routed to the Activity Listing Page (P3), while administrators are routed to the Admin Dashboard (P7).
- From the Activity Listing Page, a user can drill into Activity Details & Registration (P4), or navigate to Participation History (P5) and Schedule & Announcements (P6).
- From the Admin Dashboard, an admin can reach Manage Activities (P8) → Add/Edit Activity (P9), Manage Schedules (P10), and View Registrations & Reports (P11).
- Both modules provide a Logout action that returns the user to the Login Page.

B.2 Database Design

Five (5) related tables have been identified, exceeding the minimum of four (4) required by the assignment. Users, Activities, and Schedules were suggested in the brief; Registrations acts as the link table between Users and Activities, and Announcements was added to support the "view announcements" requirement.

Table	Key Fields	Purpose
Users	user_id (PK), name, email, password_hash, phone, role, created_at	Stores all system accounts (residents and administrators); role distinguishes access level.
Activities	activity_id (PK), title, description, category, capacity, status, created_by (FK → Users), created_at	Stores each community activity/programme; created_by links to the admin who added it.
Registrations	registration_id (PK), user_id (FK → Users), activity_id (FK → Activities), registration_date, status	Junction table resolving the many-to-many relationship between Users and Activities; also serves as the participation history record.
Schedules	schedule_id (PK), activity_id (FK → Activities), session_date, start_time, end_time, venue	Stores one or more scheduled sessions for an activity (supports recurring/multi-session activities).
Announcements	announcement_id (PK), posted_by (FK → Users), title, content, posted_date	Stores centre notices shown on the Schedule & Announcements page.

B.2.1 Entity Relationship Diagram (ERD)

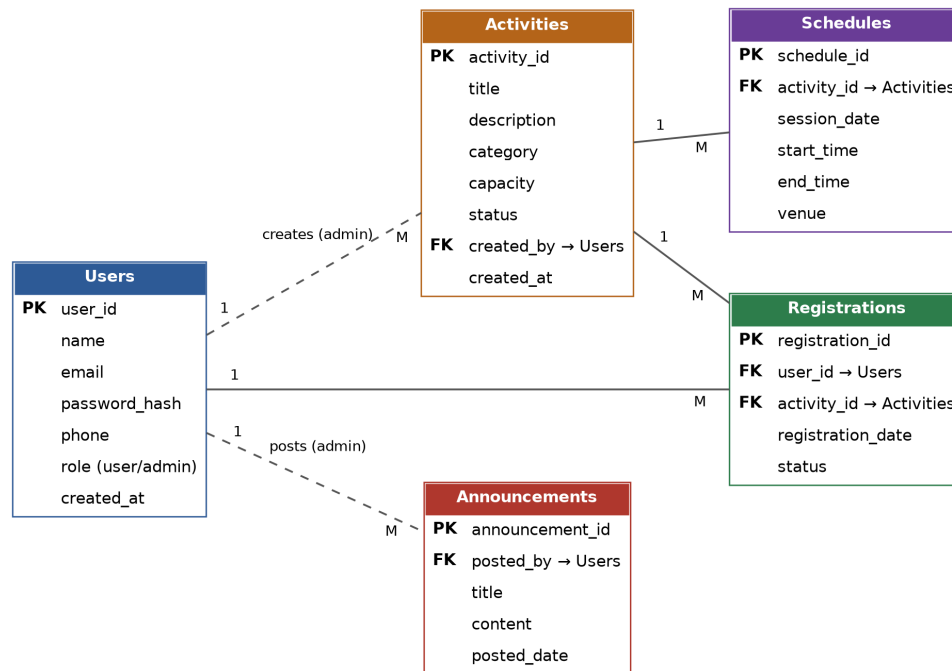


Figure 3: Entity Relationship Diagram of the Smart Community Service Management System

How the tables are connected:

- **Users – Registrations (1 : M)**: One user can make many registrations, but each registration belongs to exactly one user. `user_id` in Registrations is a foreign key referencing Users.
- **Activities – Registrations (1 : M)**: One activity can have many registrations, but each registration is for exactly one activity. `activity_id` in Registrations is a foreign key referencing Activities. Together, Registrations resolves the many-to-many relationship between Users and Activities and also functions as each user's participation history.
- **Activities – Schedules (1 : M)**: One activity can have multiple scheduled sessions (e.g., a recurring weekly workshop); each schedule row belongs to one activity via `activity_id`.
- **Users – Activities (1 : M, admin only)**: Each activity is created by one administrator (`created_by`), while one administrator may create many activities.
- **Users – Announcements (1 : M, admin only)**: Each announcement is posted by one administrator (`posted_by`), while one administrator may post many announcements.

This design keeps each table focused on a single entity, avoids data duplication, and enforces referential integrity through primary and foreign keys — consistent with the three-tier, relational (MySQL) design proposed in Part A. (If the team instead selects Firebase, the same entities and relationships would be implemented as top-level collections/sub-collections with equivalent reference fields in place of foreign keys.)